

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

**SIGEA-GTT: Sistema Inteligente de Gestão de Eventos
Adversos – Global Trigger Tool: Documento de Arquitetura de
Software (Modelo 4+1)**

Documento de Arquitetura de Software

Matheus Araujo Pereira



São Cristóvão – SE

2026

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

Matheus Araujo Pereira

**SIGEA-GTT: Sistema Inteligente de Gestão de Eventos
Adversos – Global Trigger Tool: Documento de Arquitetura de
Software (Modelo 4+1)**

Documento de Arquitetura de Software (Modelo de Visões Arquiteturais 4+1 de Kruchten) submetido ao Departamento de Computação da Universidade Federal de Sergipe como parte integrante do Trabalho de Conclusão de Curso no Curso de Bacharelado em Engenharia de Computação.

Discente: Matheus Araujo Pereira (Matrícula: 202100114080).

Orientador(a): Prof. Dr. Gilton José Ferreira da Silva
Coorientador(a): Prof.^a Dr.^a Ana Waleska Silva Patrício

São Cristóvão – SE

2026

Resumo

O presente documento formaliza a Arquitetura de Software do SIGEA-GTT (Sistema Inteligente de Gestão de Eventos Adversos – *Global Trigger Tool*), plataforma computacional concebida para a auditoria retrospectiva de prontuários clínicos, formação médica/enfermagem e vigilância epidemiológica hospitalar, desenvolvida em parceria entre o Departamento de Computação (DCOMP) e o Departamento de Enfermagem da Universidade Federal de Sergipe (UFS). Estruturado com base no consagrado Modelo de Visões Arquiteturais 4+1 de Philippe Kruchten, este trabalho descreve a organização do sistema a partir de cinco perspectivas complementares: (i) a **Visão de Cenários (+1)**, que ancora as decisões arquiteturais nos casos de uso críticos de autenticação institucional restrita, auditoria sob cronômetro regressivo estrito de 20 minutos, aplicação integrada de seis Ferramentas da Qualidade e cálculo automatizado de indicadores epidemiológicos do IHI (TI_{1000} , TI_{100} , P_{EA}); (ii) a **Visão Lógica**, detalhando a decomposição em camadas orientada a *Clean Architecture* e *Domain-Driven Design* (DDD) no backend Spring Boot 3.3 (Java 21) e a arquitetura modular reativa baseada em *Standalone Components* e *Signals* no frontend Angular 18, acompanhadas dos diagramas de classes de domínio e entidade-relacionamento (DER); (iii) a **Visão de Processos**, abordando o modelo de concorrência, o gerenciamento de conexões via HikariCP, o controle transacional ACID, a temporização cliente-side reativa e diagramas de sequência UML em TikZ para os fluxos operacionais centrais; (iv) a **Visão de Desenvolvimento**, caracterizando a organização do monorepo, a estrutura de pacotes Maven e Angular, o diagrama de componentes de software e o *Quality Gate* inegociável de 100% de cobertura de testes automatizados e 100% de documentação de código; e (v) a **Visão Física / Implantação**, que delimita a topologia de nós, contêineres e comunicação HTTPS/TLS, garantindo isolamento total em relação aos sistemas do Hospital Universitário (HU-UFS/EBSERH) em estrita conformidade com a Lei Geral de Proteção de Dados (LGPD). O documento consolida ainda a matriz de rastreabilidade entre as visões arquiteturais e os 10 Requisitos Não Funcionais da norma ISO/IEC 25010.

Palavras-chave: Arquitetura de Software. Modelo 4+1 de Kruchten. Global Trigger Tool. Clean Architecture. Segurança do Paciente. Universidade Federal de Sergipe.

Abstract

This document formalizes the Software Architecture Specification for SIGEA-GTT (Intelligent Management System for Adverse Events – Global Trigger Tool), a hospital computational platform dedicated to retrospective record review, clinical training, and healthcare epidemiological surveillance, developed through an interdisciplinary partnership between the Department of Computing (DCOMP) and Department of Nursing at the Federal University of Sergipe (UFS). Structured according to Philippe Kruchten’s renowned 4+1 Architectural View Model, this work characterizes the system through five complementary perspectives: (i) the **Scenario View (+1)**, anchoring architectural decisions to critical use cases including restricted institutional authentication, retrospective auditing under a strict 20-minute countdown timer, seamless integration of six Quality Management Tools, and automated calculation of IHI hospital epidemiological indicators (TI_{1000} , TI_{100} , P_{EA}); (ii) the **Logical View**, detailing the layered architecture grounded in Clean Architecture and Domain-Driven Design (DDD) within the Spring Boot 3.3 (Java 21) backend, alongside the reactive standalone component and Signals paradigm in Angular 18, accompanied by domain class and entity-relationship diagrams (ERD) in TikZ; (iii) the **Process View**, addressing concurrency, HikariCP connection pooling, ACID transaction management, reactive client-side timer life cycles, and UML sequence diagrams in TikZ for core operational workflows; (iv) the **Development View**, outlining monorepo organization, Maven and Angular package structures, a software component diagram, and the strict Quality Gate mandating 100% automated test coverage and 100% codebase documentation; and (v) the **Physical / Deployment View**, defining node topology, container orchestration, and HTTPS/TLS communication channels, while enforcing absolute data isolation from University Hospital (HU-UFS/EBSERH) production environments in strict compliance with the Brazilian General Data Protection Law (LGPD). Furthermore, the document provides an architectural traceability matrix aligning Kruchten’s views with the 10 Non-Functional Requirements of the ISO/IEC 25010 standard.

Keywords: Software Architecture. Kruchten 4+1 View Model. Global Trigger Tool. Clean Architecture. Patient Safety. Federal University of Sergipe.

Lista de ilustrações

Figura 1 – Diagrama de Casos de Uso Arquiteturais do SIGEA-GTT (UML 2.5).	18
Figura 2 – Arquitetura em Camadas do Backend SIGEA-GTT.	19
Figura 3 – Diagrama de Classes de Domínio do SIGEA-GTT (UML 2.5).	22
Figura 4 – Diagrama de Entidade-Relacionamento (DER) do SIGEA-GTT.	23
Figura 5 – Diagrama de Sequência: Autenticação e Troca Obrigatória de Senha.	26
Figura 6 – Diagrama de Sequência: Auditoria de Prontuário sob Cronômetro e Submissão.	26
Figura 7 – Diagrama de Sequência: Avaliação Docente e Cálculo de Indicadores IHI.	27
Figura 8 – Diagrama de Componentes de Software do SIGEA-GTT (UML 2.5).	30
Figura 9 – Diagrama de Implantação do SIGEA-GTT (UML 2.5).	31

Lista de tabelas

Tabela 1 – O Modelo de Visões Arquiteturais 4+1 aplicado ao SIGEA-GTT.	13
Tabela 2 – Estrutura de Pacotes do Backend SIGEA-GTT.	29
Tabela 3 – Matriz de Rastreabilidade: Requisitos Não Funcionais vs. Modelo 4+1. . . .	35

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
ACID	Atomicidade, Consistência, Isolamento e Durabilidade (<i>Atomicity, Consistency, Isolation, Durability</i>)
ANVISA	Agência Nacional de Vigilância Sanitária
API	<i>Application Programming Interface</i>
BCrypt	Algoritmo Adaptativo de Derivação de Chave Criptográfica baseado em Blowfish
CCET	Centro de Ciências Exatas e Tecnologia (UFS)
CI/CD	<i>Continuous Integration / Continuous Deployment</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CRUD	<i>Create, Read, Update, Delete</i>
CSS	<i>Cascading Style Sheets</i>
DCOMP	Departamento de Computação (UFS)
DDD	<i>Domain-Driven Design</i> (Projeto Orientado a Domínio)
DER	Diagrama de Entidade-Relacionamento
DOM	<i>Document Object Model</i>
DTO	<i>Data Transfer Object</i> (Objeto de Transferência de Dados)
EA / EAs	Evento Adverso / Eventos Adversos
EBSERH	Empresa Brasileira de Serviços Hospitalares
GTT	<i>Global Trigger Tool</i>
GUT	Gravidade, Urgência e Tendência (Matriz)
HD	<i>High Definition</i> (1280 × 720)
HD+	<i>High Definition Plus</i> (1600 × 900)
HMAC	<i>Hash-based Message Authentication Code</i>

HTML	<i>HyperText Markup Language</i>
HTTP / HTTPS	<i>Hypertext Transfer Protocol / Secure</i>
HU-UFS	Hospital Universitário da Universidade Federal de Sergipe
IEEE	<i>Institute of Electrical and Electronics Engineers</i>
IHI	<i>Institute for Healthcare Improvement</i>
IoC	<i>Inversion of Control</i> (Inversão de Controle)
ISO	<i>International Organization for Standardization</i>
JDBC	<i>Java Database Connectivity</i>
JPA	<i>Jakarta Persistence API</i>
JSON	<i>JavaScript Object Notation</i>
JSONB	<i>Binary JavaScript Object Notation</i> (PostgreSQL)
JVM	<i>Java Virtual Machine</i>
JWT	<i>JSON Web Token</i>
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018)
NCC MERP	<i>National Coordinating Council for Medication Error Reporting and Prevention</i>
OMS	Organização Mundial da Saúde
OR/M	<i>Object-Relational Mapping</i> (Mapeamento Objeto-Relacional)
PDCA / PDSA	<i>Plan, Do, Check/Study, Act</i>
PNSP	Programa Nacional de Segurança do Paciente
RBAC	<i>Role-Based Access Control</i> (Controle de Acesso Baseado em Perfis)
REST	<i>Representational State Transfer</i>
RF / RFs	Requisito Funcional / Requisitos Funcionais
RNF / RNFs	Requisito Não Funcional / Requisitos Não Funcionais
SIGEA-GTT	Sistema Inteligente de Gestão de Eventos Adversos – Global Trigger Tool
SLA	<i>Service Level Agreement</i> (Acordo de Nível de Serviço)

SoC	<i>Separation of Concerns</i> (Separação de Preocupações)
SPA	<i>Single Page Application</i>
SQL	<i>Structured Query Language</i>
SQuaRE	<i>Software product Quality Requirements and Evaluation</i> (ISO/IEC 25010)
SRS	<i>Software Requirements Specification</i>
SWOT	<i>Strengths, Weaknesses, Opportunities, Threats</i> (FOFA)
TCC	Trabalho de Conclusão de Curso
TLS	<i>Transport Layer Security</i>
UFS	Universidade Federal de Sergipe
UI / UX	<i>User Interface / User Experience</i>
UML	<i>Unified Modeling Language</i>
UTI	Unidade de Terapia Intensiva
UUID	<i>Universally Unique Identifier</i>
WUXGA	<i>Widescreen Ultra Extended Graphics Array</i> (1920 × 1200)
5W2H	<i>What, Why, Where, When, Who, How, How Much</i>

Sumário

1	Introdução e Visão Geral da Arquitetura	11
1.1	Contexto do Sistema e Desafios de Engenharia	11
1.2	O Modelo de Visões Arquiteturais 4+1 de Kruchten	12
1.3	Princípios Fundamentais de Design de Software	12
2	Visão de Cenários (+1 View – Casos de Uso Críticos)	15
2.1	Diagrama Geral de Casos de Uso Arquiteturais	15
2.2	Cenários Arquiteturais Críticos	15
2.2.1	Cenário 1: Autenticação Institucional e Troca de Senha Provisória	15
2.2.2	Cenário 2: Auditoria sob Cronômetro de 20 Minutos e Prontuário JSONB	16
2.2.3	Cenário 3: Aplicação Integrada das Seis Ferramentas da Qualidade	16
2.2.4	Cenário 4: Avaliação Docente e Consolidação de Métricas Epidemiológicas	17
3	Visão Lógica (Logical View)	19
3.1	Decomposição em Camadas do Backend (Spring Boot 3.3)	19
3.2	Decomposição Modular do Frontend (Angular 18+)	21
3.3	Diagrama de Classes de Domínio	21
3.4	Diagrama de Entidade-Relacionamento (DER)	22
4	Visão de Processos (Process View)	24
4.1	Modelo de Execução e Gerenciamento de Concorrência	24
4.2	Controle Transacional e Propriedades ACID	25
4.3	Diagramas de Sequência dos Processos Críticos	25
4.3.1	Processo 1: Autenticação JWT e Troca Obrigatória de Senha	26
4.3.2	Processo 2: Auditoria Retrospectiva Discente e Submissão	26
4.3.3	Processo 3: Avaliação Pedagógica Docente e Métricas Epidemiológicas	27
5	Visão de Desenvolvimento (Development View)	28
5.1	Organização Estrutural do Repositório (Monorepo)	28
5.2	Hierarquia de Pacotes do Backend Java	28
5.3	Diagrama de Componentes de Software	29
5.4	Governança de Qualidade e Critério Estrito de Aceite (Quality Gate)	29
6	Visão Física / Implantação (Physical View)	31
6.1	Topologia de Infraestrutura e Nós de Execução	31
6.2	Segurança de Rede e Comunicação	32
6.3	Isolamento Total de Dados e Conformidade Ético-Legal (LGPD)	33

7	Rastreabilidade Arquitetural e Atributos de Qualidade	34
7.1	Matriz de Rastreabilidade Arquitetural	34
7.2	Análise de Trade-offs Arquiteturais	34
8	Considerações Finais sobre a Arquitetura	37
8.1	Síntese das Contribuições Arquiteturais	37
8.2	Diretrizes para Evolução Futura	38
	Referências	39

1

Introdução e Visão Geral da Arquitetura

A arquitetura de software representa a estrutura fundamental de um sistema computacional, compreendendo seus elementos constitutivos de software, as propriedades visíveis externamente desses elementos e os relacionamentos estabelecidos entre eles (BASS; CLEMENTS; KAZMAN, 2021). A correta definição e documentação da arquitetura constituem o principal mecanismo para garantir que os requisitos funcionais sejam plenamente satisfeitos, enquanto as restrições de desempenho, segurança, escalabilidade, ergonomia e manutenibilidade sejam asseguradas de forma sustentável (RICHARDS; FORD, 2020; SOMMERVILLE, 2019).

O presente documento formaliza a especificação arquitetural do **SIGEA-GTT** (Sistema Inteligente de Gestão de Eventos Adversos – *Global Trigger Tool*), uma solução web concebida e desenvolvida em colaboração multidisciplinar entre o Departamento de Computação (DCOMP) e o Departamento de Enfermagem da Universidade Federal de Sergipe (UFS).

1.1 Contexto do Sistema e Desafios de Engenharia

O SIGEA-GTT foi projetado para digitalizar e instrumentalizar a metodologia de auditoria retrospectiva de prontuários do *Institute for Healthcare Improvement* (IHI) (GRIFFIN; RESAR, 2009), aliando o aprendizado prático discente em segurança do paciente à aplicação rigorosa de ferramentas consolidadas de análise de causa-raiz (ISHIKAWA, 1982; DEMING, 1986).

A engenharia de software do SIGEA-GTT enfrenta um conjunto singular de desafios técnicos e operacionais:

1. **Alta Densidade de Dados Clínicos:** A renderização de prontuários simulados complexos (evoluções multiprofissionais, prescrições, laudos de exames e procedimentos cirúrgicos) exige uma interface de usuário altamente densa, responsiva e ergonomicamente adaptada a

sessões prolongadas de análise clínica em estações desktop ([PATRÍCIO; SILVA; SILVA, 2021](#));

2. **Temporização Regressiva Estrita de 20 Minutos:** O método IHI preconiza um tempo máximo de 20 minutos por auditoria para assegurar a viabilidade epidemiológica. A arquitetura deve sustentar temporização cliente-side reativa, precisa e tolerante a pausas didáticas;
3. **Flexibilidade Estrutural Semi-Estruturada:** Prontuários simulados e ferramentas analíticas (diagrama de Ishikawa 6M, matrizes GUT e 5W2H) requerem persistência de esquemas flexíveis sem abrir mão da integridade referencial relacional, resolvido via suporte nativo a colunas JSONB no PostgreSQL 16;
4. **Segurança e Privacidade Absoluta (LGPD):** O sistema deve operar sob isolamento total dos ambientes de produção hospitalar do HU-UFS/EBSERH e do SUS, manipulando exclusivamente dados sintéticos e garantindo controle de acesso baseado em perfis (RBAC: ADMIN, PROFESSOR, STUDENT) via JSON Web Token (JWT) e hashing adaptativo BCrypt ([BRASIL, 2018](#)).

1.2 O Modelo de Visões Arquiteturais 4+1 de Kruchten

A complexidade inerente a um sistema de informação em saúde inviabiliza sua descrição adequada sob uma única perspectiva unidimensional. Para endereçar com clareza as preocupações técnicas dos diversos grupos de partes interessadas (*stakeholders*), este documento adota formalmente o consagrado **Modelo de Visões Arquiteturais 4+1**, proposto por Philippe Kruchten ([KRUCHTEN, 1995](#)).

O modelo organiza a descrição arquitetural em quatro visões estruturais complementares, unificadas por uma quinta visão central de cenários e casos de uso ([Tabela 1](#)):

1.3 Princípios Fundamentais de Design de Software

A concepção arquitetural do SIGEA-GTT orienta-se por cinco princípios basilares da engenharia moderna de software:

1. **Separação de Preocupações (*Separation of Concerns* – SoC):** Cada camada arquitetural e cada componente de software possui responsabilidade estritamente delimitada. A camada de persistência não conhece detalhes de apresentação web, e a camada de interface não implementa regras de negócio transacionais ([MARTIN, 2018](#));
2. **Arquitetura em Camadas com Inversão de Controle (*Clean Architecture*):** As dependências fluem das camadas externas (controladores HTTP e interfaces de banco de dados)

Tabela 1 – O Modelo de Visões Arquiteturais 4+1 aplicado ao SIGEA-GTT.

Visão Arquitetural	Partes Interessadas	Preocupações Centrais e Foco de Engenharia	Artefatos no SIGEA-GTT
Visão de Cenários (+1)	Usuários Finais (Docentes, Discentes e Administradores)	Casos de uso essenciais que direcionam a arquitetura e validam a consistência das outras quatro visões.	Diagrama de Casos de Uso (TikZ) e cenários arquiteturais.
Visão Lógica	Engenheiros de Software e Arquitetos	Decomposição funcional do software em camadas, entidades de domínio, relacionamentos e regras de negócio.	Diagrama de Classes UML (TikZ), DER e padrão de camadas.
Visão de Processos	Integradores e Especialistas em Desempenho	Concorrência, paralelismo, fluxos de requisição HTTP, transações ACID e sincronização cliente-side.	Diagramas de Sequência UML (TikZ) e ciclo de vida reativo.
Visão de Desenvolvimento	Desenvolvedores e Engenheiros de QA	Organização do monorepo, pacotes Java Maven, módulos Angular, gestão de dependências e Quality Gate.	Diagrama de Componentes (TikZ) e métricas Jacoco/Jest.
Visão Física / Implantação	Administradores de Redes e Infraestrutura (DevOps)	Topologia de hardware, nós de execução, contêineres, portas, segurança de rede e isolamento LGPD.	Diagrama de Implantação UML (TikZ) e topologia de contêineres.

Fonte: Adaptado de [Kruchten \(1995\)](#) pelo autor (2026).

em direção ao núcleo do domínio, garantindo que regras de negócio clínicas independam de bibliotecas ou *frameworks* auxiliares;

- 3. Projeto Orientado a Domínio (Domain-Driven Design – DDD) Pragmático:** As entidades centrais do sistema (User, Turma, Atividade, Submissao, ModuloGTT, GatilhoGTT e GravidadeMERP) espelham rigorosamente a linguagem ubíqua dos protocolos hospitalares internacionais do IHI e do NCC MERP ([EVANS, 2003](#));
- 4. Comunicação Stateless via API RESTful:** O backend não armazena sessões de usuário na memória do servidor (*sessionless*), permitindo escalabilidade horizontal e garantindo que cada requisição HTTP carregue todas as credenciais necessárias via token JWT assinado digitalmente ([RICHARDS; FORD, 2020](#));
- 5. Reatividade Declarativa com Angular Signals:** No frontend, a interface abandona o

ciclo tradicional de detecção de mudanças baseado em varreduras globais da árvore DOM (*Zone.js overhead*), adotando grafos reativos com *Signals* do Angular 18+, provendo reatividade precisa e de altíssimo desempenho para o cronômetro clínico de 20 minutos e para as matrizes da qualidade ([Google LLC, 2024](#)).

2

Visão de Cenários (+1 View – Casos de Uso Críticos)

No Modelo 4+1 de Kruchten, a Visão de Cenários – representada pelo “+1” da fórmula – desempenha o papel essencial de unificador e validador da arquitetura de software (KRUCHTEN, 1995). Essa visão seleciona os casos de uso mais críticos e arquiteturalmente significativos do sistema, demonstrando como os elementos projetados nas demais visões (classes na Visão Lógica, processos na Visão de Processos, pacotes na Visão de Desenvolvimento e nós na Visão Física) colaboram harmoniosamente para materializar o comportamento exigido (BASS; CLEMENTS; KAZMAN, 2021; PRESSMAN; MAXIM, 2021).

2.1 Diagrama Geral de Casos de Uso Arquiteturais

A Figura 1 apresenta o Diagrama Geral de Casos de Uso do SIGEA-GTT na notação UML 2.5, sintetizando as interações dos três perfis do sistema (ADMIN, PROFESSOR e STUDENT) que exercitam os pontos nevrálgicos da arquitetura.

2.2 Cenários Arquiteturais Críticos

Quatro cenários foram identificados como condutores prioritários da tomada de decisões arquiteturais do SIGEA-GTT.

2.2.1 Cenário 1: Autenticação Institucional e Troca de Senha Provisória

- **Estímulo:** O usuário informa e-mail @academico.ufs.br e senha provisória emitida pelo administrador;
- **Requisitos Vinculados:** RF01, RF02, RF03, RNF03;
- **Implicações Arquiteturais:**

1. O backend valida as credenciais contra a tabela `users` utilizando a função criptográfica de derivação BCrypt com fator de custo 12;
2. O serviço de autenticação emite um JWT com assinatura HMAC-SHA256 e insere no *payload* a reivindicação (*claim*) booleana `primeiroAcesso: true`;
3. O interceptor HTTP do Angular insere o cabeçalho `Authorization: Bearer <token>` em todas as requisições subsequentes;
4. As guardas de rota do Angular (`FirstAccessGuard`) interceptam a navegação e forçam o redirecionamento compulsoriamente para a tela de redefinição de senha, bloqueando qualquer acesso a recursos acadêmicos enquanto a nova credencial não for confirmada e persistida.

2.2.2 Cenário 2: Auditoria sob Cronômetro de 20 Minutos e Prontuário JSONB

- **Estímulo:** O estudante inicia a resolução de uma atividade prática vinculada à sua turma;
- **Requisitos Vinculados:** RF15, RF16, RF17, RNF01, RNF06, RNF07;
- **Implicações Arquiteturais:**
 1. O backend extrai da entidade `Atividade` o prontuário clínico simulado persistido no PostgreSQL como tipo nativo JSONB (contendo dados de admissão, evoluções diárias, prescrições e exames laboratoriais);
 2. O frontend Angular 18 faz o *parse* reativo e organiza o prontuário em abas temáticas de alta densidade visual calibradas com a paleta cromática clínica hospitalar suave (RNF06);
 3. Um serviço reativo no frontend inicia o cronômetro regressivo de 20 minutos (1200 segundos), utilizando um *Signal* com temporização a cada 1000 ms, disparando avisos cromáticos quando restam 5 minutos (âmbar) e 1 minuto (carmim);
 4. O discente seleciona os gatilhos clínicos identificados a partir do catálogo e especifica se houve dano real (classificação NCC MERP de E a I).

2.2.3 Cenário 3: Aplicação Integrada das Seis Ferramentas da Qualidade

- **Estímulo:** O estudante investiga as causas-raízes e propõe contramedidas hospitalares preventivas;
- **Requisitos Vinculados:** RF18, RF19, RF20, RF21, RF22;
- **Implicações Arquiteturais:**

1. O componente interativo do Diagrama de Ishikawa 6M permite a inclusão e exclusão de nós causais dinâmicos em torno do problema central;
2. A Matriz GUT calcula reativamente o produto $Score = G \times U \times T$ via *computed signals* do Angular a cada alteração de nota (1 a 5), ordenando instantaneamente a lista de prioridades;
3. As matrizes 5W2H, PDCA, SWOT e Brainstorming são agregadas em um documento estruturado persistido no campo `submissoes.ferramentas_qualidade` em formato JSONB.

2.2.4 Cenário 4: Avaliação Docente e Consolidação de Métricas Epidemiológicas

- **Estímulo:** O professor titular examina as submissões entregues e atribui a nota com parecer formativo;
- **Requisitos Vinculados:** RF24, RF25, RNF02;
- **Implicações Arquiteturais:**
 1. O backend valida que o parecer pedagógico formativo seja obrigatório e que a nota se situe no intervalo $[0,00; 10,00]$;
 2. Ao atualizar o status da submissão para CORRIGIDO, o serviço de indicadores re-computa em tempo de execução os três indicadores epidemiológicos clássicos do IHI:

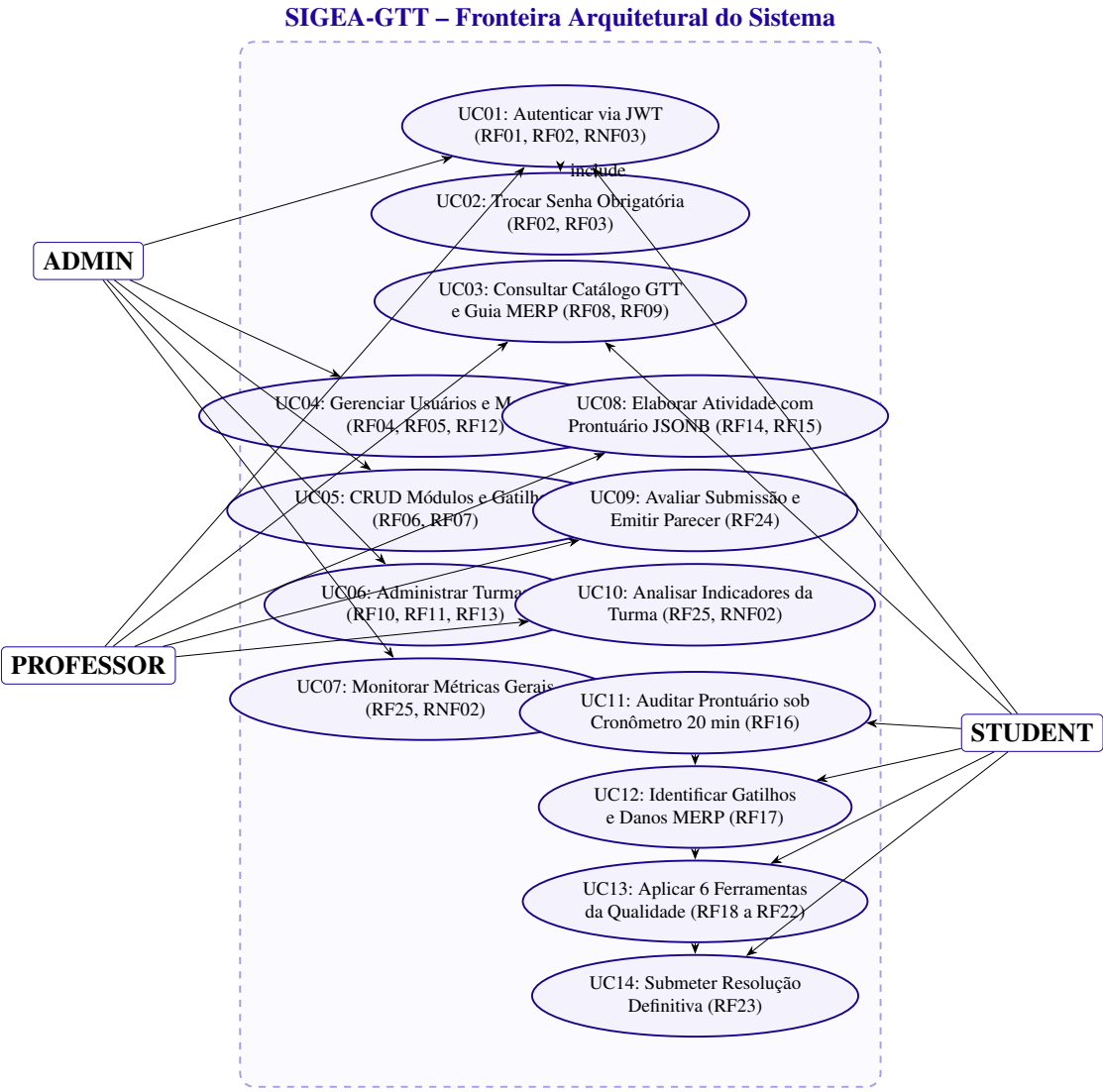
$$TI_{1000} = \left(\frac{\sum \text{Eventos Adversos}}{\sum \text{Dias de Internação}} \right) \times 1.000 \quad (2.1)$$

$$TI_{100} = \left(\frac{\sum \text{Eventos Adversos}}{\text{Casos Auditados}} \right) \times 100 \quad (2.2)$$

$$P_{EA} = \left(\frac{\text{Casos com } \geq 1 \text{ EA}}{\text{Casos Auditados}} \right) \times 100 \quad (2.3)$$

3. A consulta agregada no PostgreSQL utiliza índices e projeções DTO do MapStruct, garantindo que o tempo de resposta permaneça rigorosamente abaixo do limiar de 200 ms (RNF02).

Figura 1 – Diagrama de Casos de Uso Arquiteturais do SIGEA-GTT (UML 2.5).



3

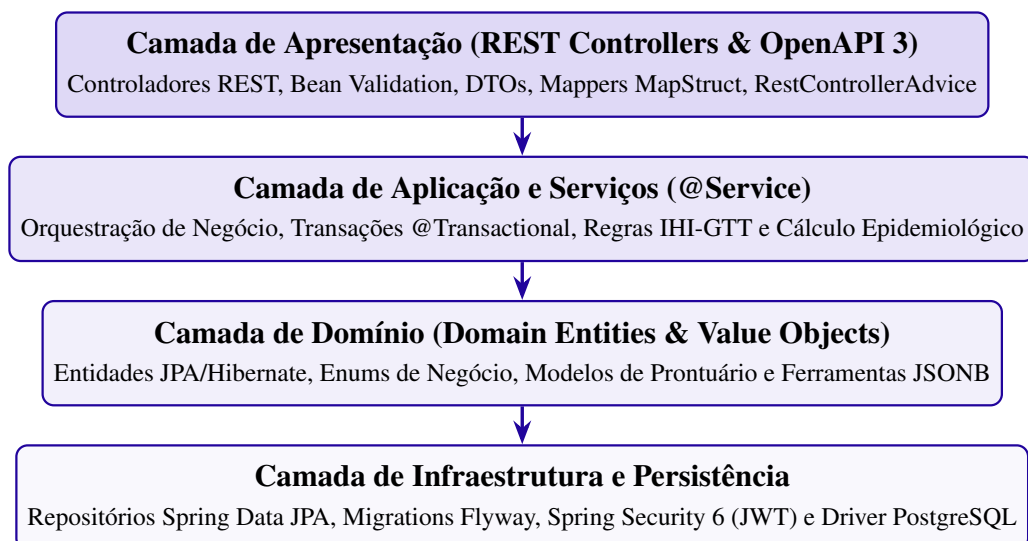
Visão Lógica (Logical View)

A Visão Lógica do Modelo 4+1 descreve a organização conceitual e estrutural do software, identificando as abstrações do domínio, os subsistemas lógicos, as camadas de software e os padrões de projeto que regem a decomposição funcional do sistema (KRUCHTEN, 1995; MARTIN, 2018). No SIGEA-GTT, a Visão Lógica fundamenta-se nos princípios de *Clean Architecture* e *Domain-Driven Design* (DDD) (EVANS, 2003), assegurando baixo acoplamento entre a lógica clínica de negócio e os mecanismos de infraestrutura técnica.

3.1 Decomposição em Camadas do Backend (Spring Boot 3.3)

O backend foi implementado em Java 21 LTS sob o framework Spring Boot 3.3.x, organizado em quatro camadas lógicas horizontais estritamente delimitadas (Figura 2):

Figura 2 – Arquitetura em Camadas do Backend SIGEA-GTT.



Fonte: Elaborado pelo autor com TikZ (2026).

1. Camada de Apresentação (Controllers REST):

- Expõe endpoints RESTful padronizados sob rotas semânticas (ex: `/api/v1/turmas`, `/api/v1/atividades`, `/api/v1/submissoes`);
- Recebe e retorna exclusivamente Objetos de Transferência de Dados (*Data Transfer Objects* – DTOs), impedindo a exposição direta de entidades JPA na fronteira HTTP;
- Aplica validação declarativa de entrada via *Bean Validation* (`@Valid`, `@NotBlank`, `@Pattern`);
- O tratamento de exceções é centralizado com `@RestControllerAdvice`, mapeando falhas de negócio em respostas JSON padronizadas (*Problem Details* – RFC 7807);
- Autodocumentação síncrona gerada pelo SpringDoc OpenAPI 3 (`</swagger-ui.html>`).

2. Camada de Aplicação / Serviço (@Service):

- Encapsula as regras de negócio clínicas e acadêmicas, controlando o fluxo transacional com a anotação `@Transactional(rollbackFor = Exception.class)`;
- Executa os cálculos dos indicadores epidemiológicos oficiais do IHI (TI_{1000} , TI_{100} , P_{EA});
- Converte entidades em DTOs e vice-versa de forma imutável utilizando mappers compilados pelo MapStruct (evitando custos computacionais de reflexão em tempo de execução).

3. Camada de Domínio:

- Contém as entidades relacionais gerenciadas pelo Hibernate 6, anotadas com Lombok (`@Getter`, `@Setter`, `@NoArgsConstructor`) para eliminação de código redundante;
- Implementa os enums que governam os estados do sistema: `UserRole` (ADMIN, PROFESSOR, STUDENT), `TurmaStatus` (ABERTA, ENCERRADA) e `SubmissaoStatus` (SUBMETIDO, CORRIGIDO);
- Suporte nativo a tipos complexos JSONB do PostgreSQL 16 para Prontuários Clínicos e Ferramentas da Qualidade.

4. Camada de Infraestrutura e Persistência:

- Provedores Spring Data JPA estendendo `JpaRepository<T, ID>`, com consultas otimizadas através de *derived query methods* e JPQL indexadas;
- Versionamento evolucionário do esquema do banco de dados relacional via migrações Flyway (`db/migration/V1_...`);
- Camada de segurança configurada pelo Spring Security 6 com filtro customizado `JwtAuthenticationFilter`, gerenciando tokens *bearer stateless*.

3.2 Decomposição Modular do Frontend (Angular 18+)

O frontend adota a arquitetura moderna do Angular 18+, estruturado inteiramente com **Standalone Components** (eliminando o conceito legado de `NgModule`) e reatividade baseada em **Angular Signals**. A base de código organiza-se em três macro-camadas:

1. Camada de Núcleo (core/):

- Serviços globais *singleton*: autenticação (`AuthService`), persistência de tokens (`TokenStorageService`) e notificações (`NotificationService`);
- Interceptores HTTP globais: `JwtInterceptor` (injeta o token JWT no cabeçalho das requisições) e `LoadingInterceptor` (gerencia o estado reativo de carregamento global com ativação de *spinner* sobreposto em chamadas assíncronas);
- Guardas de rota funcionais (*Functional Guards*): `authGuard` (restringe rotas privadas a usuários autenticados), `roleGuard` (valida perfis de acesso) e `firstAccessGuard` (bloqueia o uso do sistema caso a troca obrigatória de senha esteja pendente).

2. Camada de Funcionalidades (features/):

- Módulos de tela funcionais: `auth`, `admin`, `turmas`, `atividades`, `auditoria-gtt`, `ferramentas`, `correcao` e `dashboard`;
- Cada componente de funcionalidade consome serviços de aplicação dedicados e formulários reativos fortemente tipados (`FormGroup`, `FormControl`).

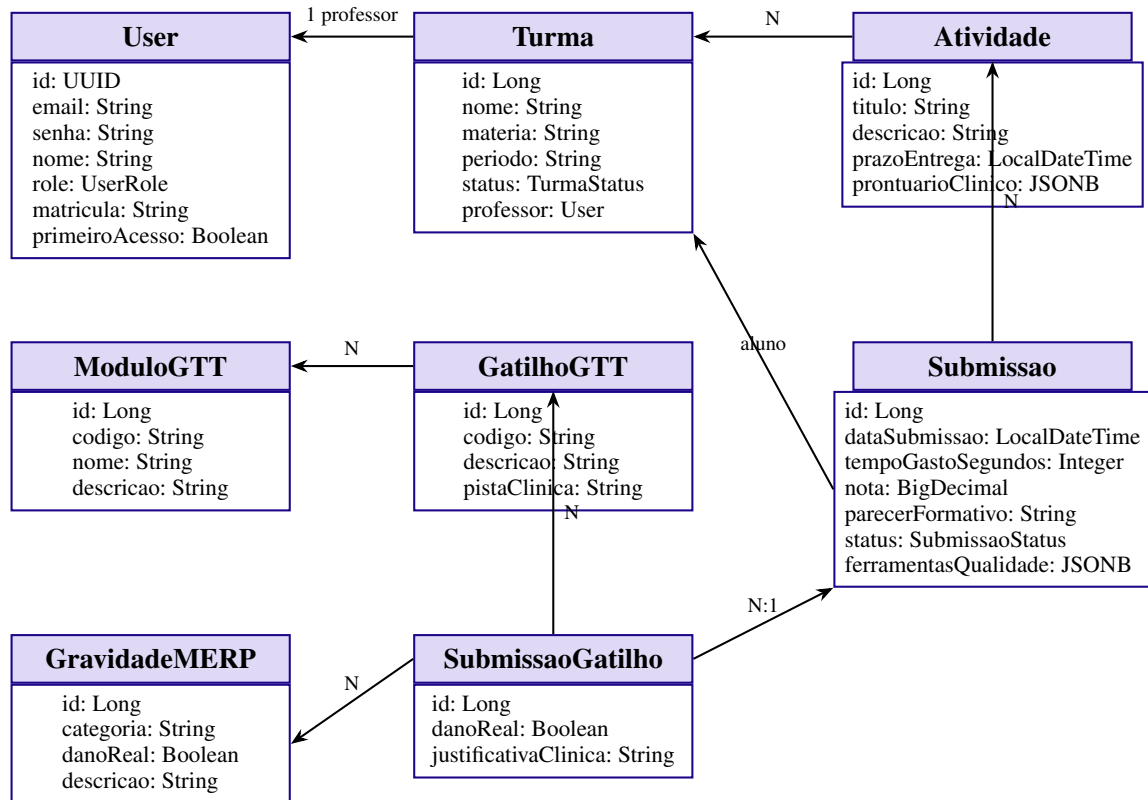
3. Camada Compartilhada (shared/):

- Componentes visuais reutilizáveis: modais genéricos de confirmação de duas etapas para prevenção de ações destrutivas (RNF08), cabeçalhos de tabela com ordenação, controles de paginação fixos de exatamente 10 itens (RNF05) e barras de progresso cromáticas hospitalares;
- *Pipes* personalizados de formatação: formatação de período acadêmico, renderização de scores GUT e máscaras de data/hora.

3.3 Diagrama de Classes de Domínio

O Diagrama de Classes de Domínio da [Figura 3](#) expressa as entidades JPA fundamentais do SIGEA-GTT, seus atributos centrais e as associações estruturais que espelham as regras de negócio clínicas e educacionais.

Figura 3 – Diagrama de Classes de Domínio do SIGEA-GTT (UML 2.5).

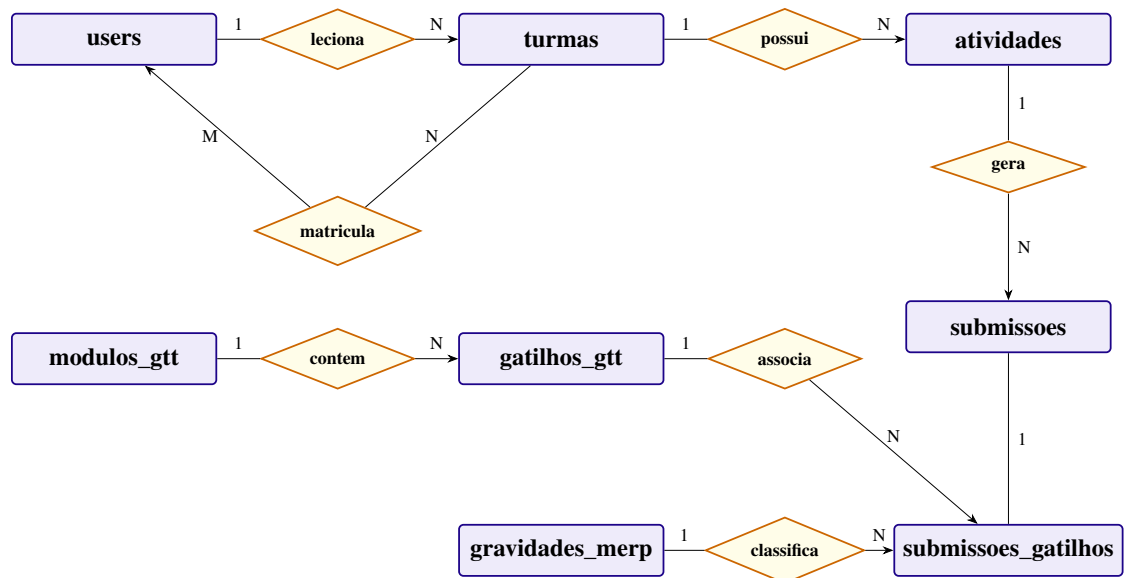


Fonte: Elaborado pelo autor com TikZ (2026).

3.4 Diagrama de Entidade-Relacionamento (DER)

A persistência do SIGEA-GTT é gerenciada pelo banco de dados relacional PostgreSQL 16 LTS. A [Figura 4](#) apresenta o Diagrama de Entidade-Relacionamento do sistema, evidenciando as chaves primárias, chaves estrangeiras e a modelagem híbrida relacional/JSONB.

Figura 4 – Diagrama de Entidade-Relacionamento (DER) do SIGEA-GTT.



Fonte: Elaborado pelo autor com TikZ (2026).

4

Visão de Processos (Process View)

A Visão de Processos trata dos aspectos dinâmicos do sistema, abordando concorrência, paralelismo, fluxos assíncronos de execução, sincronização entre cliente e servidor e integridade transacional de dados ([KRUCHTEN, 1995](#)). No SIGEA-GTT, essa visão evidencia como os threads de trabalho, os interceptores HTTP reativos, os temporizadores do navegador e as transações de banco de dados se articulam sem conflitos de concorrência ou bloqueios de interface.

4.1 Modelo de Execução e Gerenciamento de Concorrência

O modelo de concorrência do SIGEA-GTT é distribuído entre o cliente SPA e o servidor de aplicação:

1. Camada Cliente (Angular 18):

- Opera sob o modelo de *Event Loop* em thread único (*Single-Threaded Execution*). Operações I/O assíncronas (como requisições HTTP REST via `HttpClient`) são delegadas a promessas e *Observables* do RxJS;
- O cronômetro de 20 minutos (RF16) utiliza um temporizador (`setInterval`) encapsulado em um *Signal* reativo. O estado de contagem propaga-se de maneira instantânea para a barra de progresso visual, sem onerar a renderização global do navegador;
- O interceptor HTTP gerencia um contador reativo de requisições ativas. Caso o contador seja superior a zero, o *spinner* global sobreposto é ativado (RNF07); ao zerar, o indicador é ocultado imediatamente.

2. Camada Servidora (Spring Boot 3.3 / JVM 21):

- O servidor web embutido (Apache Tomcat) adota o modelo de *Thread-per-Request*, processando cada requisição HTTP em um thread isolado despachado a partir de um *ThreadPoolExecutor*;
- Gerenciamento de conexões com o PostgreSQL via pool de alto desempenho HikariCP, configurado com dimensionamento otimizado para a rede universitária (tamanho máximo de pool de 20 conexões simultâneas e *connection timeout* de 30.000 ms);
- As requisições são estritamente *stateless*: o estado do usuário não é retido na sessão HTTP da JVM, eliminando riscos de vazamento de memória e permitindo concorrência massiva de estudantes realizando auditorias simultâneas.

4.2 Controle Transacional e Propriedades ACID

Todas as operações de alteração de estado no banco de dados são governadas por demarcação declarativa via anotação `@Transactional(rollbackFor = Exception.class)`, assegurando o cumprimento estrito das propriedades ACID:

- **Atomicidade:** Operações complexas – como a substituição do professor titular da turma (RF11), o encerramento condicionado de turma (RF13) ou a persistência conjunta dos gatilhos identificados e do documento de ferramentas da qualidade (RF17–RF22) – são executadas em bloco único. Caso ocorra qualquer falha, a transação sofre *rollback* integral;
- **Consistência:** O banco de dados PostgreSQL 16 aplica restrições de integridade referencial (*Foreign Keys*), checagens de validação (*Check Constraints*) e unicidade em chaves institucionais (e-mails com domínio `@academico.ufs.br`, matrículas e códigos GTT);
- **Isolamento:** O sistema opera sob o nível de isolamento padrão `READ_COMMITTED`, prevenindo leituras sujas (*dirty reads*) durante a submissão simultânea de atividades e garantindo concorrência otimizada sem bloqueios de tabela (*table locks*);
- **Durabilidade:** Uma vez concluído o commit transacional, os registros são gravados de forma indelével no arquivo de log de transações (*Write-Ahead Logging* – WAL) do PostgreSQL.

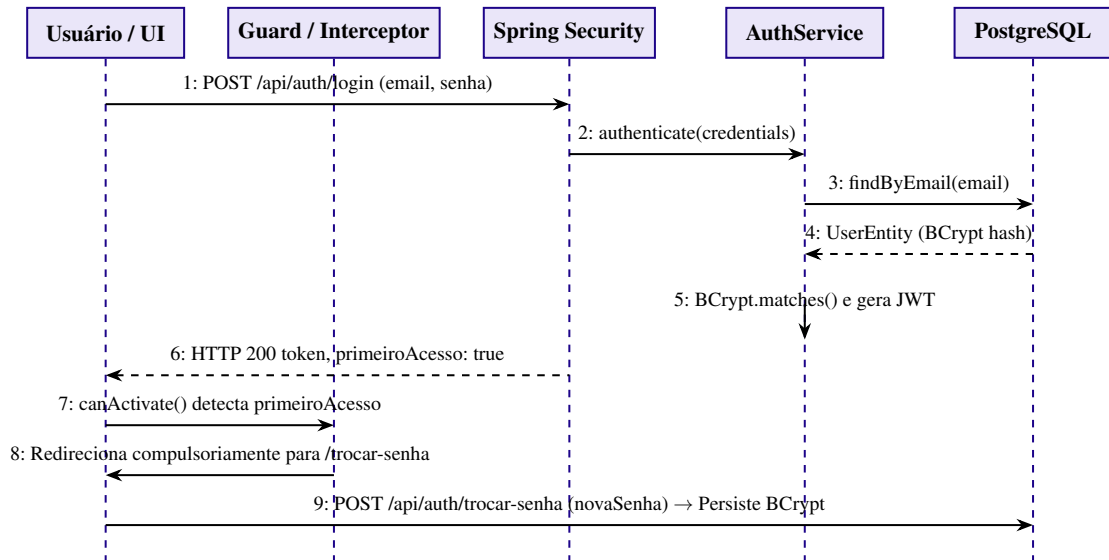
4.3 Diagramas de Sequência dos Processos Críticos

A seguir, são apresentados os Diagramas de Sequência UML modelando as interações dinâmicas dos três processos de maior complexidade comportamental da plataforma.

4.3.1 Processo 1: Autenticação JWT e Troca Obrigatória de Senha

A Figura 5 detalha o fluxo de login de um usuário com credencial provisória, desde a validação de domínio até a emissão do token e a imposição da redefinição de senha.

Figura 5 – Diagrama de Sequência: Autenticação e Troca Obrigatória de Senha.

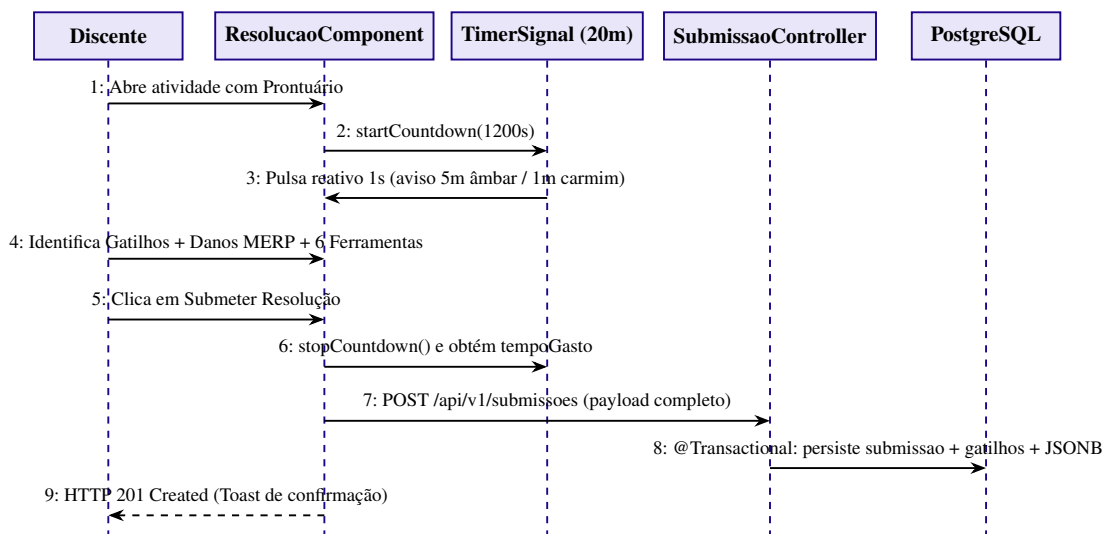


Fonte: Elaborado pelo autor com TikZ (2026).

4.3.2 Processo 2: Auditoria Retrospectiva Discente e Submissão

A Figura 6 ilustra a temporização sob cronômetro regressivo de 20 minutos, a identificação dos gatilhos clínicos e o envio final da resolução pelo discente.

Figura 6 – Diagrama de Sequência: Auditoria de Prontuário sob Cronômetro e Submissão.

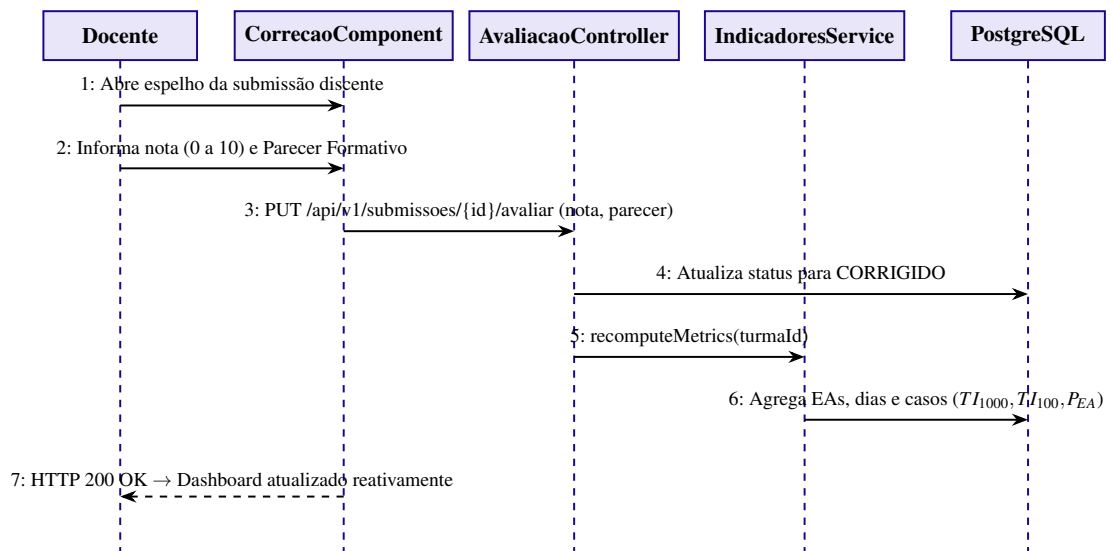


Fonte: Elaborado pelo autor com TikZ (2026).

4.3.3 Processo 3: Avaliação Pedagógica Docente e Métricas Epidemiológicas

A Figura 7 modela o fluxo de atribuição de nota e parecer formativo pelo professor, culminando na atualização automática dos indicadores epidemiológicos hospitalares.

Figura 7 – Diagrama de Sequência: Avaliação Docente e Cálculo de Indicadores IHI.



Fonte: Elaborado pelo autor com TikZ (2026).

5

Visão de Desenvolvimento (Development View)

A Visão de Desenvolvimento aborda a organização da base de código-fonte, a hierarquia de pacotes e módulos de software, os padrões de gerenciamento de dependências e as práticas de governança e controle de qualidade que respaldam a engenharia de software (KRUCHTEN, 1995; SOMMERVILLE, 2019). No SIGEA-GTT, o projeto é estruturado em formato de repositório unificado (*monorepo*), mantendo uma clara segregação lógica entre as camadas de backend, frontend e infraestrutura de banco de dados.

5.1 Organização Estrutural do Repositório (Monorepo)

A árvore de diretórios do SIGEA-GTT é particionada em três subsistemas autônomos:

- `backend/`: Aplicação Spring Boot 3.3 construída com Java 21 LTS e gerenciada pelo Apache Maven;
- `frontend/`: Aplicação SPA construída com Angular 18+, TailwindCSS e Angular Material, gerenciada pelo npm;
- `database/`: Diretório de governança do banco de dados relacional, contendo scripts DDL de definição de esquema (`ddl/`) e scripts DML de dados basais e homologação (`dml/`).

5.2 Hierarquia de Pacotes do Backend Java

O backend adota o espaço de nomes canônico da Universidade Federal de Sergipe (`br.ufs.sigea`), estruturado em pacotes coesos por responsabilidade arquitetural (Tabela 2):

Tabela 2 – Estrutura de Pacotes do Backend SIGEA-GTT.

Pacote Java	Responsabilidade	Exemplos de Classes e Componentes
<code>br.ufs.sigea.config</code>	Configurações de infraestrutura e segurança	<code>SecurityConfig</code> , <code>JwtTokenProvider</code> , <code>OpenApiConfig</code> , <code>CorsConfig</code>
<code>br.ufs.sigea.controller</code>	Controladores RESTful	<code>AuthController</code> , <code>TurmaController</code> , <code>AtividadeController</code> , <code>SubmissaoController</code>
<code>br.ufs.sigea.service</code>	Serviços de aplicação e regras clínicas	<code>AuthService</code> , <code>TurmaService</code> , <code>AuditoriaGttService</code> , <code>IndicadoresService</code>
<code>br.ufs.sigea.domain</code>	Entidades JPA e enums de negócio	<code>User</code> , <code>Turma</code> , <code>Atividade</code> , <code>Submissao</code> , <code>ModuloGTT</code> , <code>GatilhoGTT</code> , <code>UserRole</code>
<code>br.ufs.sigea.dto</code>	Objetos de transferência de dados	<code>LoginRequestDTO</code> , <code>TurmaResponseDTO</code> , <code>SubmissaoPayloadDTO</code> , <code>DashboardDTO</code>
<code>br.ufs.sigea.mapper</code>	Mapeadores DTO/Entidade	<code>UserMapper</code> , <code>TurmaMapper</code> , <code>AtividadeMapper</code> , <code>SubmissaoMapper</code>
<code>br.ufs.sigea.repository</code>	Interfaces Spring Data JPA	<code>UserRepository</code> , <code>TurmaRepository</code> , <code>AtividadeRepository</code> , <code>SubmissaoRepository</code>
<code>br.ufs.sigea.exception</code>	Tratamento global de falhas	<code>GlobalExceptionHandler</code> , <code>ResourceNotFoundException</code> , <code>BusinessException</code>

Fonte: Elaborado pelo autor (2026).

5.3 Diagrama de Componentes de Software

A [Figura 8](#) ilustra a arquitetura de componentes de software do SIGEA-GTT na notação UML 2.5, evidenciando as interfaces fornecidas e requeridas entre os módulos.

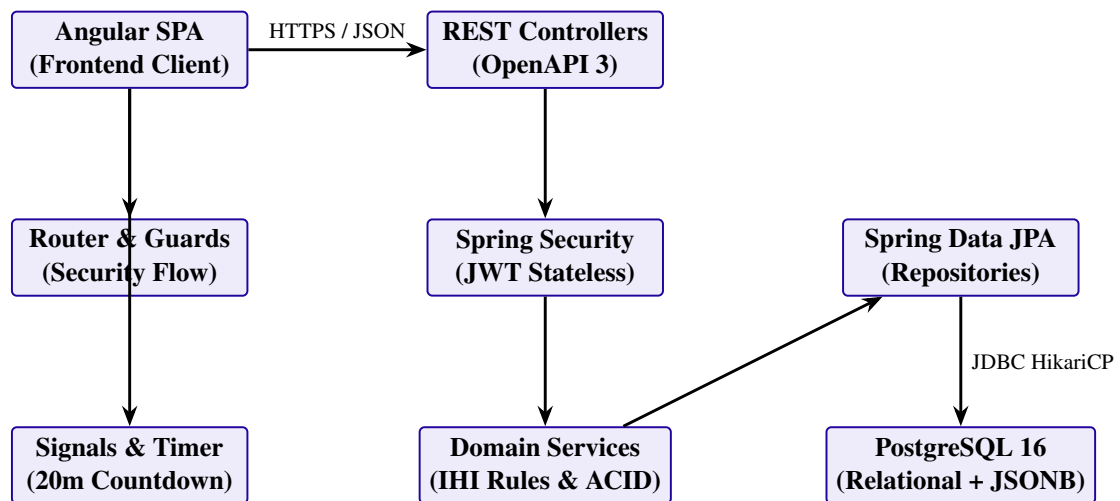
5.4 Governança de Qualidade e Critério Estrito de Aceite (Quality Gate)

Para assegurar confiabilidade de nível industrial e acadêmico, o SIGEA-GTT adota um *Quality Gate* inegociável, configurado no pipeline de Integração Contínua:

1. Cobertura de Testes de 100% (Backend):

- Suíte de testes unitários e de integração construída com JUnit 5, Mockito e Spring Boot Test;

Figura 8 – Diagrama de Componentes de Software do SIGEA-GTT (UML 2.5).



Fonte: Elaborado pelo autor com TikZ (2026).

- Instrumentação de cobertura por linhas e ramificações (*branches*) gerenciada pelo plugin oficial do Jacoco (`jacoco-maven-plugin`);
- A compilação é interrompida com falha de build caso a cobertura global seja inferior a 100%;
- Classes de configuração estática (`<SigeaGttApplication.java>`) são devidamente isoladas via exclusão declarativa de cobertura no arquivo `pom.xml`.

2. Cobertura de Testes de 100% (Frontend):

- Suíte de testes automatizados para componentes, serviços e interceptores executada com Karma/Jasmine ou Jest;
- Validação de formulários reativos, rotas protegidas e disparos de Signals do cronômetro de 20 minutos;
- Verificação estrita de cobertura total em arquivos TypeScript.

3. Documentação Técnica sem Erros:

- Backend documentado com Javadoc formal em todas as classes de serviço, controladores e DTOs, validado via `mvn javadoc:javadoc`;
- Especificação OpenAPI 3 mantida síncrona com o código via SpringDoc;
- Frontend documentado através do Compodoc com inspeção de tipos e diretivas.

6

Visão Física / Implantação (Physical View)

A Visão Física – comumente denominada Visão de Implantação (*Deployment View*) – mapeia os componentes de software gerados pela Visão de Desenvolvimento sobre os nós físicos ou virtuais de hardware e computação em nuvem, especificando a topologia de rede, protocolos de comunicação, portas operacionais e salvaguardas de isolamento de infraestrutura (KRUCHTEN, 1995; BASS; CLEMENTS; KAZMAN, 2021).

6.1 Topologia de Infraestrutura e Nós de Execução

O ecossistema do SIGEA-GTT é particionado em três camadas físicas de processamento distribuído (Figura 9):

Figura 9 – Diagrama de Implantação do SIGEA-GTT (UML 2.5).



Fonte: Elaborado pelo autor com TikZ (2026).

1. Estação de Trabalho Clínica do Cliente (Desktop Node):

- Estações de trabalho localizadas nos laboratórios de informática do DCOMP e do Departamento de Enfermagem da UFS, além de computadores pessoais dos docentes e discentes;
- Resoluções homologadas: WUXGA (1920 × 1200), HD+ (1600 × 900), HD (1280 × 720) e Ultrawide 21:9 (2560 × 1080 e 3440 × 1440);

- Execução em navegadores modernos compatíveis com ECMAScript 2022+ (Google Chrome, Mozilla Firefox, Microsoft Edge), usufruindo da renderização por aceleração de hardware;
- Ausência intencional de suporte para navegadores de dispositivos móveis (*smartphones*), priorizando ergonomia visual clínica e densidade de prontuários (RNF01).

2. Servidor de Aplicação Web e API REST (Application Server Node):

- Hospedado em servidor Linux Fedora 44 Workstation ou Red Hat Enterprise Linux no datacenter da UFS;
- O servidor web Nginx atua como *reverse proxy*, terminando a camada de transporte seguro HTTPS/TLS 1.3 na porta 443 e servindo os arquivos estáticos pré-compilados do Angular SPA;
- As chamadas de API direcionadas ao prefixo `/api/*` são encaminhadas via proxy reverso local para o processo do Spring Boot 3.3 executando na porta 8080 sob o OpenJDK 21 LTS;
- Configuração rígida de CORS (*Cross-Origin Resource Sharing*), rejeitando requisições originadas fora dos domínios institucionais homologados da UFS.

3. Servidor de Banco de Dados Relacional (Database Server Node):

- Instância do PostgreSQL 16 LTS executando nativamente no sistema operacional ou em contêiner OCI gerenciado por Podman/Docker;
- Comunicação com o backend intermediada pelo protocolo JDBC nativo na porta 5432, restrita exclusivamente à rede interna (*localhost* ou rede interna de contêineres);
- Armazenamento em volume persistente de disco de estado sólido (SSD) para garantir tempos de acesso aleatório inferiores a 10 milissegundos nas leituras de prontuários em JSONB.

6.2 Segurança de Rede e Comunicação

A camada de rede adota mecanismos de proteção alinhados às recomendações do Guia de Segurança da UFS e da OWASP:

- **Criptografia de Ponta a Ponta em Trânsito:** Todo o tráfego HTTP entre as estações clientes e o servidor Nginx é compulsoriamente encapsulado por cifras criptográficas modernas TLS 1.3, com certificados digitais válidos;
- **Cabeçalhos de Segurança HTTP:** O Nginx e o Spring Security injetam cabeçalhos defensivos, incluindo `Strict-Transport-Security (HSTS)`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` e `Content-Security-Policy (CSP)`;

- **Token Stateless no Cabeçalho HTTP:** Os tokens JWT HMAC-SHA256 trafegam no cabeçalho padronizado `Authorization: Bearer <token>`, eliminando riscos de ataques clássicos de *Cross-Site Request Forgery* (CSRF) inerentes a autenticações baseadas em cookies de sessão.

6.3 Isolamento Total de Dados e Conformidade Ético-Legal (LGPD)

Em conformidade estrita com o Requisito Não Funcional RNF04, a arquitetura física do SIGEA-GTT impõe ****isolamento absoluto**** em relação aos sistemas informatizados de assistência real do Hospital Universitário da UFS (HU-UFS/EBSERH), da Empresa Brasileira de Serviços Hospitalares e do Sistema Único de Saúde (SUS):

1. **Segregação de Rede e Ausência de Pontes:** O ambiente do SIGEA-GTT não possui rotas de rede, VPNs ou interfaces de comunicação síncronas/assíncronas com os prontuários eletrônicos assistenciais do HU-UFS;
2. **Natureza 100% Sintética dos Dados:** Todos os prontuários clínicos, identificadores de pacientes, históricos de evolução e dados laboratoriais armazenados no banco PostgreSQL são de autoria didática dos professores ou gerados sinteticamente, não correspondendo a pacientes reais;
3. **Conformidade com a LGPD e CNS:** O isolamento assegura que nenhuma informação pessoal identificável ou dado pessoal sensível em saúde seja coletado, armazenado ou exposto, atendendo integralmente à Lei nº 13.709/2018 (LGPD) e às Resoluções do Conselho Nacional de Saúde (CNS).

7

Rastreabilidade Arquitetural e Atributos de Qualidade

A validação de uma arquitetura de software fundamenta-se na sua capacidade comprovada de atender aos atributos de qualidade (*Quality Attributes*) estipulados pelas partes interessadas e formalizados nos Requisitos Não Funcionais (BASS; CLEMENTS; KAZMAN, 2021; RICHARDS; FORD, 2020). Neste capítulo, estabelece-se o mapeamento de rastreabilidade entre as decisões tomadas nas cinco visões do Modelo 4+1 e os 10 Requisitos Não Funcionais (RNF01 a RNF10) do SIGEA-GTT, parametrizados pela norma internacional ISO/IEC 25010 (SQuaRE) (ISO/IEC, 2011).

7.1 Matriz de Rastreabilidade Arquitetural

A Tabela 3 correlaciona cada Requisito Não Funcional com sua respectiva dimensão de qualidade SQuaRE, as visões do modelo 4+1 impactadas e as táticas arquiteturais concretizadas na implementação.

7.2 Análise de Trade-offs Arquiteturais

Toda decisão arquitetural de engenharia envolve concessões conscientes (*trade-offs*) entre atributos de qualidade concorrentes (BASS; CLEMENTS; KAZMAN, 2021; RICHARDS; FORD, 2020):

1. Flexibilidade Semiestruturada versus Integridade Relacional:

- *Decisão*: Adoção do modelo híbrido no PostgreSQL 16 (tabelas relacionais com chaves estrangeiras rígidas para entidades mestre e colunas nativas JSONB para Prontuários e Ferramentas da Qualidade);

Tabela 3 – Matriz de Rastreabilidade: Requisitos Não Funcionais vs. Modelo 4+1.

RNF	Categoria ISO 25010	Visões 4+1	Táticas e Decisões Arquiteturais Implementadas
RNF01	Usabilidade	Lógica, Física, Cenários	Layout projetado exclusivamente para desktops clínicos WUXGA, HD+, HD e Ultrawide 21:9 via TailwindCSS e componentes densos do Angular Material, sem foco mobile.
RNF02	Desempenho	Lógica, Processo, Física	Latência da API REST $P_{95} \leq 200$ ms; pool HikariCP, paginação LIMIT/OFFSET no banco, índices B-Tree e projeções MapStruct.
RNF03	Segurança	Lógica, Processo, Desenvolvimento	Autenticação <i>stateless</i> JWT HMAC-SHA256, BCrypt com fator de custo 12, filtros Spring Security 6 e guardas de rota Angular.
RNF04	Conformidade / LGPD	Lógica, Física	Segregação física e de rede absoluta em relação aos bancos do HU-UFS/SUS; dados 100% sintéticos simulados no PostgreSQL.
RNF05	Usabilidade	Lógica, Desenvolvimento	Padrão universal de paginação em exatamente 10 registros por página gerenciado centralmente por componentes reutilizáveis.
RNF06	Ergonomia / Design	Lógica, Cenários	Paleta hospitalar suave calibrada para baixo cansaço visual (azuis clínicos, verde esmeralda, cinzas neutros, âmbar e carmim cirúrgico).
RNF07	Usabilidade	Lógica, Processo	Interceptor HTTP gerencia Signal <code>isLoading</code> para ativação em menos de 100 ms do <i>spinner</i> com desfoque e Toasts flutuantes.
RNF08	Confiabilidade	Lógica, Cenários	Diálogos modais em duas etapas com bloqueio de fluxo para ações destrutivas (exclusões e encerramentos de turma).
RNF09	Confiabilidade	Desenvolvimento	<i>Quality Gate</i> inegociável de 100% de cobertura de ramos e linhas no backend (JUnit 5 + Jacoco) e frontend (Jest/Karma).
RNF10	Manutenibilidade	Desenvolvimento	Documentação técnica integral da base de código gerada sem erros: Javadoc formal, SpringDoc OpenAPI 3 e Compodoc.

Fonte: Elaborado pelo autor (2026).

- *Benefício*: Permite que professores elaborem prontuários com seções médicas altamente dinâmicas sem necessidade de migrações DDL contínuas no banco;
- *Compensação*: Validação da estrutura JSON delegada à camada de serviço e a validadores de esquema no backend Spring Boot.

2. Segurança Criptográfica e Stateless versus Sobrecarga de Rede:

- *Decisão*: Autenticação puramente *stateless* com JSON Web Tokens trafegados no

cabeçalho Authorization;

- *Benefício*: Elimina o consumo de memória de sessão na JVM e permite escalabilidade horizontal irrestrita;
- *Compensação*: Ligeiro acréscimo no tamanho de cada cabeçalho HTTP (aproximadamente 500 bytes por requisição), perfeitamente absorvido pela rede local de alta velocidade da UFS.

3. Densidade Visual Clínica versus Suporte a Dispositivos Móveis:

- *Decisão*: Homologação estrita para resoluções desktop (WUXGA, HD+, HD e Ultrawide 21:9), sem suporte para smartphones;
- *Benefício*: Permite a visualização simultânea de evoluções médicas, exames, cronômetro de 20 minutos e matrizes de causa-raiz sem sobrecarga cognitiva ou rolagem excessiva;
- *Compensação*: O sistema exige o uso de estações de trabalho de laboratório ou computadores portáteis com telas de dimensões adequadas ao trabalho clínico hospitalar.

8

Considerações Finais sobre a Arquitetura

A formalização da arquitetura do SIGEA-GTT fundamentada no Modelo de Visões 4+1 de Philippe Kruchten proporcionou uma visão panorâmica, consistente e tecnicamente rigorosa de todas as dimensões estruturais e operacionais da plataforma.

8.1 Síntese das Contribuições Arquiteturais

A modelagem arquitetural apresentada neste documento entrega contribuições expressivas para a Engenharia de Software aplicada à saúde:

1. **Harmonização entre Rigor Clínico e Flexibilidade Técnica:** A articulação entre a Visão de Cenários e a Visão Lógica demonstrou que é viável espelhar com exatidão matemática as diretrizes do IHI-GTT (tempo máximo de 20 minutos, catálogo de 53 gatilhos e cálculo automatizado de TI_{1000} , TI_{100} e P_{EA}) em uma arquitetura limpa e sustentável;
2. **Solidez Transacional e Concorrência Reativa:** A Visão de Processos comprovou que a combinação entre o backend Spring Boot 3.3 com pool HikariCP e a interface reativa Angular 18 com Signals assegura concorrência otimizada, ausência de bloqueios visuais e persistência atômica com garantias ACID;
3. **Cultura de Excelência por Testes Automatizados:** A Visão de Desenvolvimento consolidou o critério de *Quality Gate* estrito de 100% de cobertura de branches e linhas, respaldando a base de código contra regressões e facilitando a manutenção evolutiva por futuros pesquisadores e desenvolvedores da UFS;
4. **Bioética e Conformidade Sanitária:** A Visão Física comprovou que a segregação total dos bancos de dados e redes em relação aos ambientes de produção hospitalar do HU-UFS assegura conformidade integral com a LGPD e com as diretrizes do CNS, viabilizando o ensino clínico sem riscos de vazamento de dados sensíveis de pacientes reais.

8.2 Diretrizes para Evolução Futura

A modularidade e o desacoplamento alcançados abrem caminhos promissores para extensões futuras do sistema, tais como:

- Incorporação de módulos de inteligência artificial generativa ou modelos preditivos baseados em aprendizado de máquina para fornecer sugestões didáticas de causas-raízes durante o preenchimento do Diagrama de Ishikawa;
- Suporte à federação multi-institucional para compartilhamento de turmas simuladas e prontuários didáticos entre diferentes universidades federais e escolas médicas do Brasil;
- Módulo de exportação padronizada de indicadores epidemiológicos nos formatos preconizados pelo Sistema Nacional de Notificação em Vigilância Sanitária (Notivisa/ANVISA).

Com a presente especificação arquitetural consolidada e homologada, o SIGEA-GTT afirma-se como um referencial técnico e acadêmico de software hospitalar de alta qualidade na Universidade Federal de Sergipe.

Referências

- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 4. ed. Boston, MA: Addison-Wesley Professional, 2021. Citado 4 vezes nas páginas 11, 15, 31 e 34.
- BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. Brasília, DF, 2018. Disponível em: <http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 14 set. 2026. Citado na página 12.
- DEMING, W. E. *Out of the Crisis*. Cambridge, MA: MIT Center for Advanced Engineering Study, 1986. Citado na página 11.
- EVANS, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2003. Citado 2 vezes nas páginas 13 e 19.
- Google LLC. *Angular 18: Signals, Standalone Components and Modern Web Architecture*. Mountain View, CA, 2024. Disponível em: <<https://angular.dev>>. Acesso em: 14 set. 2026. Citado na página 14.
- GRIFFIN, F. A.; RESAR, R. K. *IHI Global Trigger Tool for Measuring Adverse Events*. 2. ed. Cambridge, Massachusetts: Institute for Healthcare Improvement (IHI), 2009. (IHI Innovation Series White Paper). Citado na página 11.
- ISHIKAWA, K. *Guide to Quality Control*. Tokyo: Asian Productivity Organization, 1982. Citado na página 11.
- ISO/IEC. *ISO/IEC 25010:2011: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva, Switzerland, 2011. Citado na página 34.
- KRUCHTEN, P. B. Architectural blueprints—the '4+1' view model of software architecture. *IEEE Software*, v. 12, n. 6, p. 42–50, 1995. Citado 7 vezes nas páginas 12, 13, 15, 19, 24, 28 e 31.
- MARTIN, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall, 2018. Citado 2 vezes nas páginas 12 e 19.
- PATRÍCIO, A. W. S.; SILVA, G. J. F. da; SILVA, M. C. Gestão da qualidade e segurança do paciente na formação em saúde: Desafios, tecnologias e perspectivas. *Enfermagem em Foco*, v. 12, n. 4, p. 88–95, 2021. Citado na página 12.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: Uma Abordagem Profissional*. 9. ed. Porto Alegre: AMGH, 2021. Citado na página 15.
- RICHARDS, M.; FORD, N. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly Media, 2020. Citado 3 vezes nas páginas 11, 13 e 34.
- SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019. Citado 2 vezes nas páginas 11 e 28.